

Übersetzung und Analyse von Programmen

A stylized, low-poly illustration of a university building with large windows and a red abstract sculpture. In the foreground, there are two green trees and a large yellow flower with a blue center. The sky is blue with a large yellow sun and green geometric shapes.

Vorlesung im Wintersemester 2025/26

Prof. Dr. Stephan Diehl
Informatik, Universität Trier



Übersetzung von TRIPLA

Projekt

TRAM implementieren

Lexikalische Analyse implementieren
Parser implementieren

Berechnung der Adressumgebung
Erzeugung von TRAM-Code

TRIPLA: Trierer Programmiersprache

Gegeben sei die Grammatik G:

```
E → let D in E
   | ID
   | ID ( A )
   | E AOP E
   | ( E )
   | CONST
   | ID = E
   | E ; E
   | if B then E else E
   | while B do { E }
A → E | A , E
D → ID ( V ) { E }
   | D D
V → ID | V , V
B → E | E RELOP E
```

ID: Bezeichner (fangen mit Buchstaben an und bestehen aus Buchstaben und Zahlen)

CONST: ganze Zahlen

AOP: arithm. Operatoren (+, -, *, /)

RELOP: Vergleichsoperatoren (==, !=, <, >)

Menge aller gültigen Programme: $LG(E) = \{w \mid E \rightarrow^* w\}$

Beispiel:

```
let f1(b) {
    if (b==0) then 0 else f1(b-1)
}

f2(a,b) {
    if (a>b) then f1(a) else f1(b)
}

in f1(10); f2(10,20)
```

TRIPLA: Trierer Programmiersprache

Beispiel:

DEKLARATIONEN

```
let f1(b) {  
    if (b==0) then 0 else f1(b-1)  
}  
  
f2(a,b) {  
    if (a>b) then f1(a) else f1(b);  
    let g(c) {  
        a*b*c  
    }  
    in g(a*b)  
}
```

Parameterlose Methoden
sind nicht erlaubt!

Es existiert immer
ein else-Zweig

Rückgabewert
der Funktion

Verschachtelungen
von Funktionen

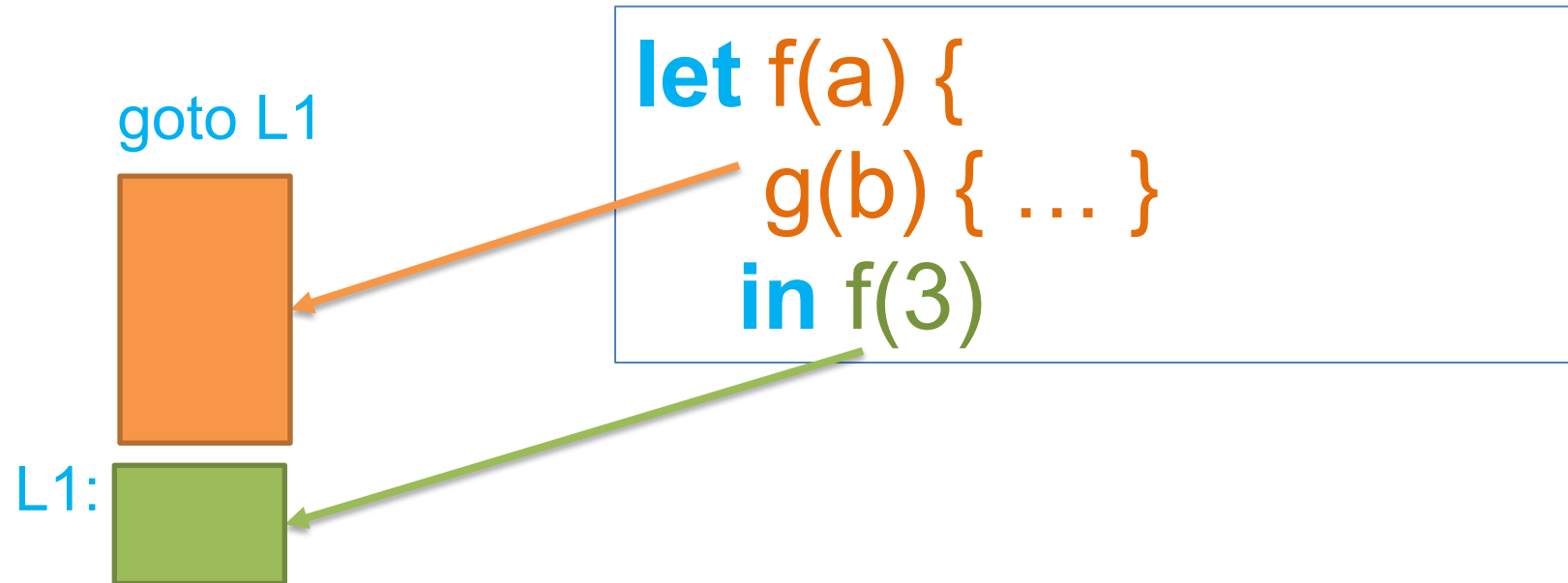
Zugriff auf Variablen
umschließender
Funktionen

AUFRUFE

```
in f1(10);  
f2(10, let max(a,b) {  
    if (a>b) then a else b  
}  
in max(20,30) )
```

Rückgabewert des
let-in-Ausdrucks

Kodeerzeugung



Kodeerzeugung

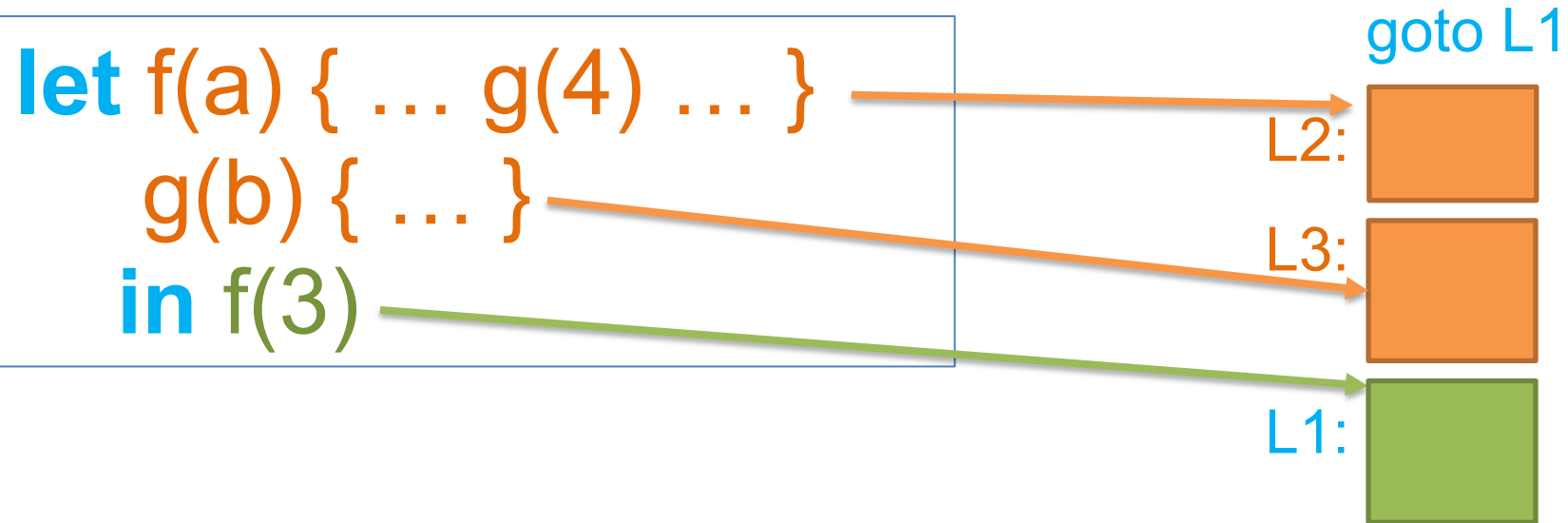


(K1) $code\ (d_1 \dots d_k)\ \rho\ nl = code\ d_1\ \rho\ nl;$
...
 $code\ d_k\ \rho\ nl$

(K2) $code\ (let\ d\ in\ e)\ \rho\ nl =$
 $goto\ \ell;$
 $code\ d\ \rho'\ nl;$
 $\ell : code\ e\ \rho'\ nl$
 where ℓ is a new label **and** $\rho' = elab_def\ d\ \rho\ nl$

Berechnung der lokalen Adressumgebung

Kodeerzeugung



Berechnung der Adressumgebung



$$(E1) \quad \textit{elab_def} (d_1 \dots d_k) \rho \textit{ nl} = \rho_k \quad \textbf{where} \quad \rho_0 = \rho \textit{ and } \rho_i = \textit{elab_def} d_i \rho_{i-1} \textit{ nl}$$

$$(E2) \quad \textit{elab_def} (\textit{id}(\textit{id}_1, \dots, \textit{id}_k)\{e\}) \rho \textit{ nl} = \rho[(\ell, \textit{nl}) \backslash \textit{id}] \quad \textbf{where } \ell \textit{ is a new label}$$

$$\text{Notation: } f[v \backslash z](x) = \begin{cases} v, & \text{falls } x = z \\ f(x), & \text{sonst} \end{cases} \quad \begin{array}{l} \text{Überschreiben der Funktion} \\ f \text{ an der Stelle } z \text{ mit dem Wert } v \end{array}$$

Kodeerzeugung

(K3) $\text{code id } \rho \text{ nl} = \text{load } k \text{ (nl-nl')}$ **where** $\rho(\text{id}) = (k, \text{nl}')$

(K4) $\text{code (id=e) } \rho \text{ nl} =$
 $\text{code e } \rho \text{ nl};$
 $\text{store } k \text{ (nl-nl')};$
 $\text{load } k \text{ (nl-nl')}$ **where** $(k, \text{nl}') = \rho(\text{id})$

(K5) $\text{code (e}_1\text{==e}_2\text{) } \rho \text{ nl} =$
 $\text{code e}_1 \rho \text{ nl};$
 $\text{code e}_2 \rho \text{ nl}; \text{eq}$

Analog für !=, <, >

(K6) $\text{code k } \rho \text{ nl} = \text{const } k$

(K7) $\text{code (e}_1\text{+e}_2\text{) } \rho \text{ nl} =$
 $\text{code e}_1 \rho \text{ nl};$
 $\text{code e}_2 \rho \text{ nl};$
 add

Analog für -, *, /

(K8) $\text{code ((e)) } \rho \text{ nl} = \text{code e } \rho \text{ nl};$

Beispiel: *code* $a=b+5$ $\{a \mapsto (0,2), b \mapsto (4,1)\}$ 3

load 4 2

const 5

add

store 0 1

load 0 1

(K3) *code* id ρ $nl = \text{load } k \text{ } (nl-nl')$ **where** $\rho(id) = (k, nl')$

(K4) *code* $(id=e)$ ρ $nl = \text{code } e \text{ } \rho \text{ } nl; \text{store } k \text{ } (nl-nl'); \text{load } k \text{ } (nl-nl')$ **where** $(k, nl') = \rho(id)$

(K6) *code* k ρ $nl = \text{const } k$

(K7) *code* (e_1+e_2) ρ $nl = \text{code } e_1 \text{ } \rho \text{ } nl; \text{code } e_2 \text{ } \rho \text{ } nl; \text{add}$

Kodeerzeugung

(K9) $code(e_1; \dots; e_k) \rho nl = code\ e_1 \rho nl; \text{pop}; \dots; \text{pop}; code\ e_k \rho nl$

(K10) $code(\text{if } e_1 \text{ then } e_2 \text{ else } e_3) \rho nl = code\ e_1 \rho nl; \text{ifzero } \ell_1; code\ e_2 \rho nl;$
 $\text{goto } \ell_2; \ell_1: code\ e_3 \rho nl; \ell_2: \text{nop}$
where ℓ_1, ℓ_2 are new labels

(K11) $code(\text{while } e_1 \text{ do } \{ e_2 \}) \rho nl = code\ e_1 \rho nl; \text{ifzero } \ell_3; \text{goto } \ell_4;$
 $\ell_1: code\ e_1 \rho nl; \text{ifzero } \ell_2; \text{pop};$
 $\ell_4: code\ e_2 \rho nl; \text{goto } \ell_1;$
 $\ell_3: \text{const null};$
 $\ell_2: \text{nop}$
where $\ell_1, \dots, \ell_4$ are new labels

Alternativen:

- **Statt null, die Zahl 0**
- Keine Sonderbehandlung, der Keller enthält dann einen Wert weniger

- Kompliziert wg. Sonderfall, dass erster Test der Schleife bereits fehlschlägt

while $a > 0$ **do** { $a = a - 1$ }



code (while e_1 do { e_2 }) ρ nl =

code e_1 ρ nl; ifzero l_3 ; goto l_4 ;

l_1 : *code* e_1 ρ nl; ifzero l_2 ; pop;

l_4 : *code* e_2 ρ nl; goto l_1 ;

l_3 : const null;

l_2 : nop

where $l_1, \dots, l_4$ are new labels

Kodeerzeugung

(K12) $\text{code } (\text{id}(e_1, \dots, e_k)) \rho \text{ nl} =$ $\text{code } e_1 \rho \text{ nl};$
...
 $\text{code } e_k \rho \text{ nl};$
 $\text{invoke } k \ell (\text{nl} - \text{nl}')$

where $\rho(\text{id}) = (\ell, \text{nl}')$

(K13) $\text{code } (\text{id}(\text{id}_1, \dots, \text{id}_k)\{e\}) \rho \text{ nl} =$
 $\ell : \text{code } e \rho [(0, \text{nl} + 1) \setminus \text{id}_1] \dots [(k-1, \text{nl} + 1) \setminus \text{id}_k] \text{ nl} + 1;$
 return

where $\rho(\text{id}) = (\ell, \text{nl}')$

Gesamter Übersetzungsvorgang

- Sei $s \in L_G(E)$ ein Programm
- Der Code wird erzeugt mit:

code $s \neq 0$; *halt*

Beispiel

Dieses Beispielprogramm terminiert nicht; dies ist aber für die Übersetzung unerheblich.

- code

```
let f(a) {  
  let g(b) { f(g(a+b)) }  
  in g(a) }  
in f(3)
```

∅ 0; halt

code let f(a) {
 let g(b) { f(g(a+b)) }
 in g(a) }
 in f(3) 0 ; halt

L1
L2
L3
L4

(K2) $\downarrow$
 goto L1; code f(a) { ... } ρ_1 0; L1: code f(3) ρ_1 0

(K12) $\rightarrow$ code 3 ρ_1 0; invoke 1 L2 0
 const 3

(K13) $\left(\begin{array}{l} \rho_1 = \{ f \vdash (L2, 0) \} \\ \downarrow \\ L2: \text{code let } g(b) \{ \dots \} \dots \rho_2 \text{ 1; return} \end{array} \right.$

$\xrightarrow{\text{load 0 0}}$ code a ρ_3 1; invoke 1 L4 0

(K2) $\swarrow$ $\rho_2 = \rho_1 \cup \{ a \vdash (0, 1) \}$
 goto L3: code g(b) { ... } ρ_3 1; L3: code g(a) ρ_3 1

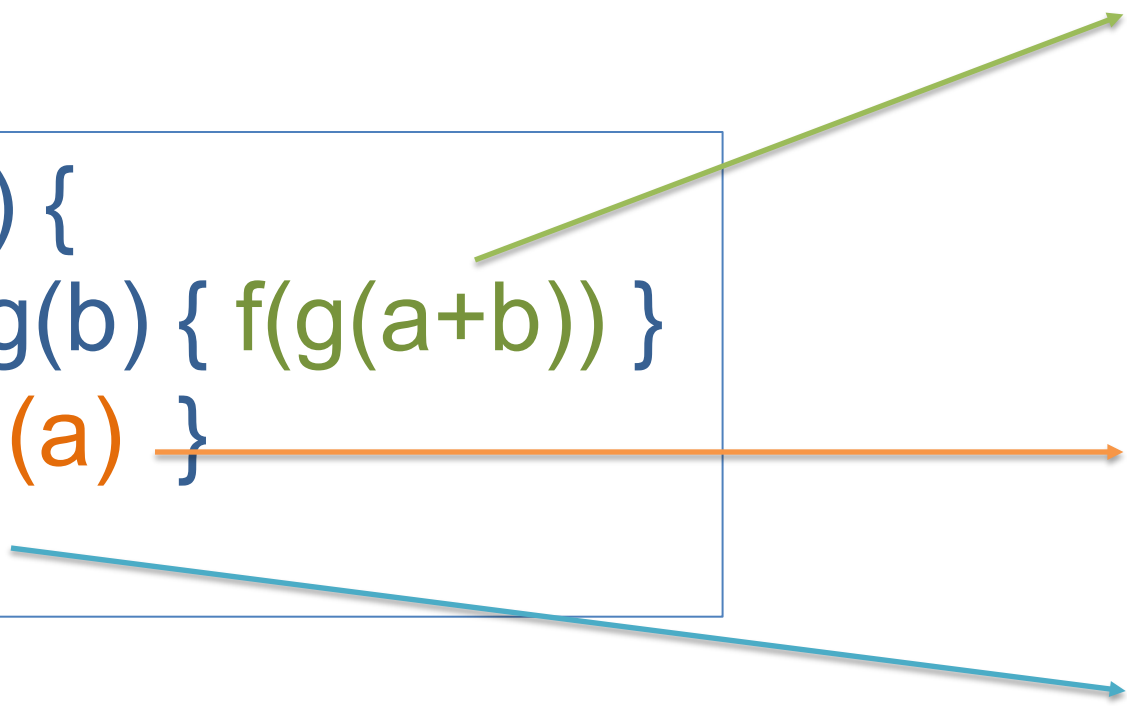
(K13) $\left(\begin{array}{l} \rho_3 = \rho_2 \cup \{ g \vdash (L4, 1) \} \\ \downarrow \\ L4: \text{code } f(g(a+b)) \rho_4 \text{ 2; return} \end{array} \right.$

(K12) $\left(\begin{array}{l} \rho_4 = \rho_3 \cup \{ b \vdash (0, 2) \} \\ \downarrow \\ \text{code } g(a+b) \rho_4 \text{ 2; invoke 1 L2 2} \end{array} \right.$

$\xrightarrow{\text{K12}}$ code a+b ρ_4 2; invoke 1 L4 1

$\rightarrow$ code a ρ_4 2; code b ρ_4 2, add
 load 0 1 load 0 0

```
let f(a) {  
  let g(b) { f(g(a+b)) }  
  in g(a) }  
in f(3)
```



goto L1
L2: goto L3
L4: load 0 1
load 0 0
add
invoke 1 L4 1
invoke 1 L2 2
return
L3: load 0 0
invoke 1 L4 0
return
L1: const 3
invoke 1 L2 0
halt